

Relatório Técnico de Resiliência

Universidade Federal do Espírito Santo
Centro de Ciências Exatas, Naturais e da Saúde — CCENS/UFES
Departamento de Computação

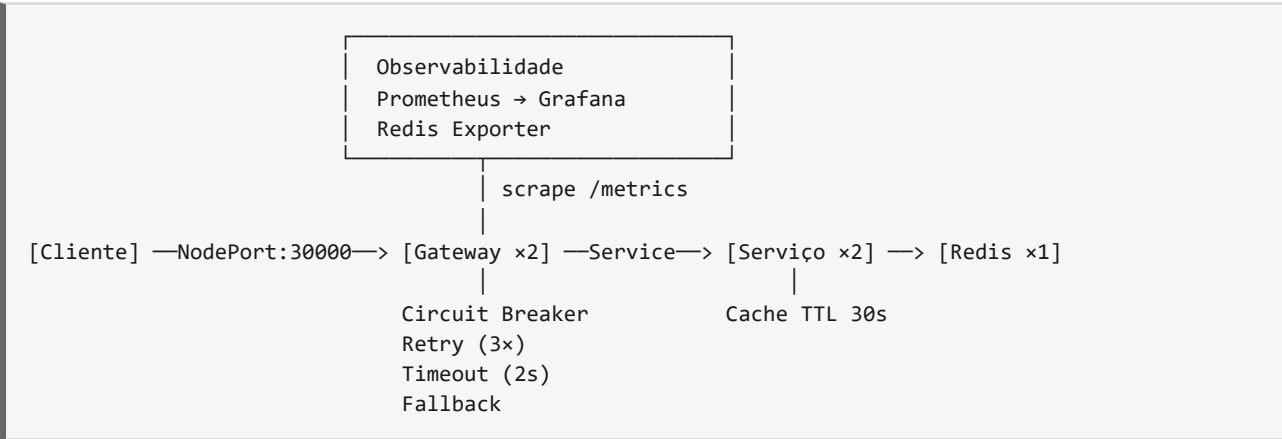
Disciplina: Sistemas Distribuídos — 2026/1

Professor: Helder de Amorim Mendes

Grupo:
Blendhon Pontini Delfino
Jhonatas Vinicius Neri Dos Santos
Maria Clara Gueler Feitani

1. Arquitetura da Aplicação

A aplicação-alvo é composta por três componentes distintos e interdependentes, todos orquestrados por Kubernetes em cluster Kind de três nós (1 control-plane + 2 workers).



Componente	Tecnologia	Réplicas	Papel
Gateway	Flask + gunicorn (2 workers)	2 (HPA 2-5)	Porta de entrada; concentra os mecanismos de tolerância a falhas
Serviço	Flask + gunicorn (2 workers)	2 (HPA 2-5)	Lógica de negócio; consulta e popula o cache
Redis	redis:7	1	Camada de cache (TTL 30s) e estado compartilhado dos <i>toggles</i>
Prometheus	prom/prometheus	1	Coleta de métricas (NodePort 30090)
Grafana	grafana/grafana	1	Visualização (NodePort 30030)
Redis Exporter	oliver006/redis_exporter	1	Métricas do Redis para o Prometheus

1.1. Mecanismos de Tolerância a Falhas Implementados

Mecanismo	Camada	Configuração	Objetivo
Circuit Breaker	Software (Gateway)	3 falhas → OPEN; recuperação 10s; HALF_OPEN → CLOSED no primeiro sucesso	Interromper cascata de falhas
Timeout	Software (Gateway)	2s por requisição ao Serviço	Limitar latência máxima
Retry	Software (Gateway)	3 tentativas, backoff linear 0.5s x tentativa	Absorver falhas transitórias
Fallback	Software (Gateway)	Resposta estática status: degraded , HTTP 200	Degradação elegante
Cache	Software (Serviço + Redis)	TTL 30s	Reduzir dependência do processamento
Réplicas	Infraestrutura (K8s)	2 pods por componente	Redundância ativa
HPA	Infraestrutura (K8s)	min=2, max=5; alvo 50% CPU / 70% memória	Escalonamento sob carga
Probes	Infraestrutura (K8s)	startup / liveness (15s) / readiness (10s)	Detecção de falha e remoção do balanceamento

Nota sobre o fallback. O fallback do Gateway (`app/gateway/app.py` , rota `/api/data`) retorna uma resposta estática de degradação com HTTP 200 — ele **não** relê o cache. Essa é uma escolha deliberada de degradação elegante: o cliente recebe 200 com `status: "degraded"` e um payload informativo, em vez de um 5xx. A limitação e a melhoria proposta estão registradas na seção 6.

2. Metodologia Experimental

Os experimentos foram executados com **Chaos Mesh v2.8.3** sobre Kind, com o `chaosDaemon` configurado para o runtime containerd. A automação está em `run-experiments.ps1`.

Protocolo por experimento: limpeza do cache Redis (quando aplicável) → aplicação do manifesto de caos → espera de estabilização (3–8s) → envio de 15 requisições ao Gateway com intervalo de 300ms → coleta de métricas → remoção do caos → 15s de recuperação.

Sequência executada: Baseline (15 req) → NetworkChaos → PodChaos → StressChaos → Recuperação (10 req).

2.1. Fontes de dados e sua confiabilidade

Esta distinção é essencial para a leitura correta dos resultados e é uma das conclusões técnicas deste trabalho.

Fonte	O que mede	Confiável?
Cliente (<code>run-experiments.ps1</code>)	<code>ok / degraded / error</code> , latência média e p95	Sim. Vem da resposta HTTP real observada pelo cliente. É a evidência primária deste relatório.
<code>kubect1 top</code> / HPA (metrics-server)	CPU e memória por pod	Sim , com ressalva de defasagem temporal (~15s de resolução).
Contadores do Prometheus (<code>gateway_errors_total</code> , <code>gateway_retries_total</code> , <code>gateway_forward_requests_total</code>)	Retries, erros, fallbacks	Não. Ver diagnóstico abaixo.

Diagnóstico — contadores do Prometheus não são confiáveis nesta implantação. A biblioteca `prometheus_client` mantém os contadores em memória **por processo**. Como cada pod roda `gunicorn --workers 2` e há 2 réplicas, existem **4 processos independentes**, cada um com seu próprio registro de métricas. Cada *scrape* do Prometheus é atendido por um worker aleatório e retorna os contadores **apenas daquele worker**. O modo `multiprocess` (`PROMETHEUS_MULTIPROC_DIR`) não está configurado.

A evidência está nos próprios dados coletados: a série `gateway_forward_requests_total{status="success"}` assume, ao longo da execução, os valores **67 → 63 → 83 → 87 → 96**. Um contador Prometheus é monotônico por definição — a queda de 67 para 63 é impossível para um contador real e comprova que scrapes sucessivos leram workers distintos. O mesmo ocorre com `errors_total{type="request_error"}` (21 → 20 → 19).

Consequência metodológica: neste relatório, os contadores do Prometheus são tratados como **indicativos**, nunca como medida de um experimento isolado. Todas as afirmações quantitativas sobre sucesso, degradação e latência derivam da medição no cliente. A correção está registrada na seção 6.

3. Experimentos de Caos

3.0. Estado Estável (Baseline)

Medição de referência, sem ataque, com 15 requisições.

Métrica	Valor
Requisições <code>ok</code>	15/15 (100%)
Requisições <code>degraded</code> / <code>error</code>	0 / 0
Latência média	5,87 ms
Latência p95	6 ms
CPU Gateway (média/pod)	1m (1% do request de 150m)
CPU Serviço (média/pod)	1m (0% do request de 150m)
Memória Gateway	71.892 Ki (54%)
Memória Serviço	61.308 Ki (46%)
Réplicas (Gateway / Serviço)	2 / 2
Circuit Breaker	CLOSED, 0 falhas

Ressalva importante sobre o estado estável. O Deployment define `PROCESSING_DELAY_MS=100`, ou seja, um processamento sem cache custa ~100ms. A latência média de 5,87ms mostra que **praticamente todas as requisições do baseline foram servidas pelo cache Redis** (TTL 30s). Portanto o "estado estável" aqui caracteriza o sistema **em regime de cache quente**, e não o custo real de processamento. Toda comparação contra este baseline deve considerar esse fator — ele é revisitado na seção 5.

3.1. Experimento 1 — Falha de Rede (NetworkChaos)

Estado Estável. 15/15 `ok`, latência média 5,87ms, p95 6ms, Circuit Breaker CLOSED, 0 falhas.

Hipótese. Com latência injetada de 3000ms (± 500 ms de jitter) somada a 30% de perda de pacotes no caminho Gateway→Serviço, e sendo o timeout do Gateway de 2s, esperava-se que **nenhuma requisição concluísse dentro do timeout**. O retry (3 tentativas) deveria falhar em todas elas, o Circuit Breaker deveria abrir após 3 falhas consecutivas e o fallback deveria assumir, retornando `degraded` com HTTP 200 — isto é, **o sistema degradaria de forma elegante, sem devolver erro ao cliente**. O cache foi limpo antes do experimento justamente para forçar as requisições a atingirem o Serviço.

Configuração do Ataque. Dois manifestos aplicados simultaneamente (`chaos/network-chaos.yaml`):

```
# 1) Latência sobre os pods do serviço
kind: NetworkChaos
spec:
  action: delay
  mode: all
  selector: { labelSelectors: { app: servico } }
  delay: { latency: "3000ms", jitter: "500ms", correlation: "50" }
  duration: "300s"
---
# 2) Perda de pacotes no caminho gateway → serviço
kind: NetworkChaos
spec:
  action: loss
  mode: all
  selector: { labelSelectors: { app: gateway } }
  loss: { loss: "30", correlation: "25" }
  target:
    mode: all
    selector: { labelSelectors: { app: servico } }
  duration: "300s"
```

Resultado Observado.

Métrica	Baseline	NetworkChaos	Variação
Requisições ok	15/15	0/15	-100%
Requisições degraded	0	15	—
Requisições error (5xx/exceção)	0	0	—
Latência média	5,87 ms	2.408,13 ms	~410×
Latência p95	6 ms	7.514 ms	~1.250×
Circuit Breaker (worker amostrado)	CLOSED, 0 falhas	CLOSED, 2 falhas	—
CPU Serviço	1m (0%)	1m (0%)	—
Réplicas	2 / 2	2 / 2	—

Dados brutos desta fase no Apêndice A.

Análise de Causalidade. A hipótese confirmou-se no aspecto central: **100% das requisições foram degradadas e 0% retornaram erro ao cliente.** O sistema não quebrou — ele degradou exatamente como projetado. O contraste entre a latência média (2.408ms) e o p95 (7.514ms) é informativo: um p95 de ~7,5s corresponde ao caminho completo de 3 tentativas com timeout de 2s mais o backoff (2s + 0,5s + 2s + 1,0s + 2s ≈ 7,5s), enquanto a média muito abaixo disso indica que **boa parte das requisições falhou rapidamente**, sem esgotar o orçamento de tempo — comportamento compatível com os 30% de perda de pacotes, que produzem erro de conexão imediato em vez de timeout.

A hipótese **não** se confirmou em um ponto: o Circuit Breaker **não abriu** (permaneceu CLOSED com 2 falhas no worker amostrado), apesar de 15 falhas consecutivas do ponto de vista do cliente. A causa está descrita abaixo e é o achado mais relevante deste experimento.

Ações Corretivas.

Já implementado e validado neste experimento:

- **Timeout de 2s** — impediu que requisições ficassem presas nos 3000ms de latência injetada.
- **Retry com backoff** — exercitado em todas as tentativas (sem sucesso aqui, por se tratar de falha persistente e não transitória, o que é o comportamento correto).
- **Fallback com HTTP 200 + status: degraded** — garantiu 0 erros ao cliente; foi o mecanismo responsável pela degradação elegante observada.

Correção identificada a partir deste experimento (proposta, não implementada):

- **O estado do Circuit Breaker precisa ser compartilhado.** O breaker é um objeto em memória (`CircuitBreaker` em `app/gateway/app.py`), e existem 4 processos (2 réplicas × 2 workers gunicorn), cada um com seu próprio `failure_count`. As 15 falhas se distribuíram entre 4 breakers independentes (~3–4 cada), de modo que o limiar efetivo de abertura passou de 3 para até 12 falhas agregadas. **O mecanismo está correto; a topologia de execução o neutraliza.** Correção: mover `state`, `failure_count` e `last_failure_time` para o Redis — que o projeto já usa para estado compartilhado dos *toggles* — ou, como paliativo, executar o Gateway com `--workers 1` e deixar a concorrência a cargo das réplicas do Kubernetes.
- **Reset do contador de falhas.** Em `CircuitBreaker.call`, `failure_count` só é zerado na transição `HALF_OPEN → CLOSED`. Um sucesso em estado `CLOSED` não zera o contador, de modo que falhas esparsas e não correlacionadas se acumulam indefinidamente e podem abrir o breaker sem que exista falha sustentada. Correção: zerar `failure_count` em todo sucesso com estado `CLOSED`.

3.2. Experimento 2 — Falha de Instância (PodChaos)

Estado Estável. 2 pods do Serviço prontos e recebendo tráfego balanceado; 15/15 `ok`; latência média 5,87ms.

Hipótese. A indisponibilidade abrupta de 1 dos 2 pods do Serviço (`mode: one`) faria com que parte das requisições fosse roteada para o pod morto e falhasse. Esperava-se que o retry reencaminhasse essas requisições para a réplica saudável e que o readiness probe removesse o pod defeituoso do balanceamento, mantendo o sistema **funcional, com penalidade apenas de latência**.

Configuração do Ataque. (`chaos/pod-chaos.yaml`)

```
kind: PodChaos
spec:
  action: pod-failure
  mode: one
  selector: { labelSelectors: { app: servico } }
  duration: "30s"
```

Resultado Observado.

Métrica	Baseline	PodChaos	Variação
Requisições <code>ok</code>	15/15	15/15	0%
Requisições <code>degraded</code> / <code>error</code>	0 / 0	0 / 0	—
Latência média	5,87 ms	46 ms	~7,8×
Latência p95	6 ms	508 ms	~85×
Circuit Breaker	CLOSED	CLOSED	—
CPU Serviço	1m (0%)	23m (15%)	~23×
Réplicas antes → depois	2 → 2	2 → 2	—

Dados brutos desta fase no Apêndice A.

Análise de Causalidade. Nenhuma requisição falhou do ponto de vista do cliente — o objetivo de disponibilidade foi plenamente atingido. A evidência mais interessante está na diferença entre a média (46ms) e o p95 (508ms), que permite reconstruir o que aconteceu com precisão:

O p95 de **508ms** é a assinatura exata do mecanismo de retry. O backoff implementado é `time.sleep(0.5 × tentativa)`, ou seja, **500ms** após a primeira tentativa falha. Uma requisição que atingiu o pod morto recebeu recusa de conexão imediata (o `pod-failure` substitui o container por um *pause*, então não há timeout — há erro instantâneo), aguardou 500ms e acertou a réplica saudável na segunda tentativa: 500ms + ~8ms de serviço ≈ 508ms. A aritmética da média confirma que isso ocorreu com **aproximadamente 1 das 15 requisições**: com ~1 requisição a 508ms, 1 a ~100ms (cache miss após o flush) e 13 a ~5ms (cache hit), a média resulta em ~45ms — praticamente os 46ms medidos.

Por que apenas ~1 requisição pagou esse custo, e não metade delas? Porque o Kubernetes removeu o pod defeituoso dos *endpoints* do Service em poucos segundos: assim que o container deixou de executar, a condição `Ready` do pod tornou-se falsa e o controlador de endpoints o retirou do balanceamento. As requisições seguintes já não foram sequer roteadas para ele. Ou seja, a resiliência observada é resultado de **três camadas cooperando**: réplicas (havia para onde ir), readiness probe (parou de mandar tráfego para o pod morto) e retry (recuperou a única requisição que pegou a janela de transição).

A elevação de CPU do Serviço (1m → 23m) reflete a concentração de todo o tráfego na réplica sobrevivente somada ao trabalho de reinicialização do container afetado.

Ações Corretivas.

Já implementado e validado neste experimento:

- **2 réplicas com Service balanceando** — forneceram destino alternativo.
- **Readiness probe (10s)** — removeu o pod defeituoso do balanceamento rapidamente, limitando o dano à janela de transição.
- **Retry com backoff de 500ms** — recuperou a requisição afetada; é o mecanismo responsável pelo p95 de 508ms e por 15/15 `ok`.

Melhorias identificadas (propostas, não implementadas):

- `PodDisruptionBudget` com `minAvailable: 1`, protegendo contra a remoção simultânea de todas as réplicas em operações de manutenção (drain/upgrade), cenário que o `mode: one` não cobre.
- Elevar `replicas` para 3, preservando redundância mesmo com um pod fora.
- Cobrir também `action: pod-kill` — ver "Ameaças à validade" (seção 5), item 1.

3.3. Experimento 3 — Falha de Recurso (StressChaos)

Estado Estável. CPU do Serviço em 1m (0% do request de 150m); latência média 5,87ms; 2 réplicas.

Hipótese. A saturação de CPU (2 workers a 100% de carga) nos pods do Serviço causaria contenção de recursos, elevando a latência de processamento; o HPA deveria detectar a ultrapassagem do alvo de 50% de utilização e **escalar o Serviço de 2 para até 5 réplicas**.

Configuração do Ataque. (`chaos/stress-chaos.yaml`)

```
kind: StressChaos
spec:
  mode: all
  selector: { labelSelectors: { app: servico } }
  stressors:
    cpu: { workers: 2, load: 100 }
  duration: "45s"
```

Resultado Observado.

Métrica	Baseline	StressChaos	Recuperação (pós-ataque)
Requisições <code>ok</code>	15/15	15/15	10/10
Requisições <code>degraded</code> / <code>error</code>	0 / 0	0 / 0	0 / 0
Latência média	5,87 ms	165,2 ms	5,4 ms
Latência p95	6 ms	2.165 ms	6 ms
CPU Serviço (via HPA)	1m (0%)	27m (18%)	248m (165%)
CPU dos pods do Serviço (via <code>container_cpu_usage_seconds_total</code>)	~1m	79m e 47m	—
Réplicas antes → depois	2 → 2	2 → 2	2
Circuit Breaker	CLOSED	CLOSED	CLOSED

Dados brutos desta fase, incluindo o CPU por pod medido via `container_cpu_usage_seconds_total`, no Apêndice A.

Análise de Causalidade. O ataque **foi efetivamente injetado**: o `container_cpu_usage_seconds_total` registrou os dois pods do Serviço em **79m e 47m** durante o ataque, contra ~1m no baseline, e a latência média subiu 28× (5,87ms → 165ms). O sistema permaneceu funcional: 15/15 `ok`, embora o p95 de 2.165ms mostre que a requisição mais lenta ficou no limite do timeout de 2s.

O HPA não escalou — mas não pela razão que a leitura ingênua sugere. A medição do HPA durante o ataque (27m, 18%) está **abaixo** do alvo de 50%, o que levaria a concluir que a CPU nunca subiu o suficiente. Essa conclusão é contrariada pelo dado seguinte: na fase de **recuperação**, já com o StressChaos removido, o HPA registrou **248m (165%)** — mais de três vezes o alvo. A CPU do ataque só apareceu na média do HPA **depois que o ataque terminou**.

A causa é a defasagem do pipeline de métricas: o metrics-server tem resolução de ~15s, o HPA consulta em intervalos de ~15s e opera sobre uma média móvel, e o `scaleUp.stabilizationWindowSeconds` está em 30s. O ataque durou **45s**, e o script mediu o HPA por volta de 20s após a injeção — cedo demais para que a média móvel, ainda contaminada pelos zeros anteriores, cruzasse o limiar. Quando o sinal finalmente atravessou o pipeline, o estressor já não existia. Em resumo: **o HPA não falhou; ele nunca teve tempo de ver a carga**. A janela de observação foi menor que a latência do sistema de observação.

Há ainda um fator de projeto que reduz a severidade do ataque: o "processamento" do Serviço é `time.sleep(delay)`, e `time.sleep` não consome CPU — libera o processador. Um estressor de CPU tem, por construção, pouco a disputar com um processo que dorme. Isso é discutido na seção 5.

Ações Corretivas.

Já implementado e validado neste experimento:

- **Limites de recursos (`limits.cpu: 300m`)** — confinaram o estressor ao cgroup do container, impedindo que a saturação contaminasse os demais pods do nó. O Gateway permaneceu em 1,3m durante o ataque, e Prometheus, Grafana e Redis não foram afetados. **Este é o mecanismo que efetivamente conteve a falha**, e a evidência é o isolamento observado.
- **Timeout + retry** — mantiveram 15/15 `ok` apesar da latência 28× maior.

Correções identificadas a partir deste experimento (propostas, não implementadas):

- **Duração do ataque maior que a latência do pipeline de métricas.** Elevar `duration` de 45s para ≥180s, de modo que a média móvel do HPA cruze o limiar ainda durante o ataque. Esta é a correção que efetivamente permitiria demonstrar o autoscaling.
 - **Medir o HPA ao longo do tempo, não em um instante.** O script coleta o HPA uma única vez; amostrar a cada 5s durante todo o ataque tornaria a defasagem visível em vez de enganosa.
 - **Reduzir `requests.cpu` de 150m para ~50m**, tornando o HPA proporcionalmente mais sensível.
 - **Substituir `time.sleep` por trabalho de CPU real** (p. ex., laço de `hashlib.sha256` calibrado para ~100ms), para que o experimento meça contenção de CPU de fato. Ver seção 5.
-

4. Resultados Consolidados

4.1. Comportamento observado pelo cliente (evidência primária)

Experimento	ok	degraded	error	Latência média	p95	Réplicas	Veredito
Baseline	15/15	0	0	5,87 ms	6 ms	2/2	Estado estável
NetworkChaos	0/15	15	0	2.408,13 ms	7.514 ms	2/2	Degradação elegante
PodChaos	15/15	0	0	46 ms	508 ms	2/2	Transparente ao cliente
StressChaos	15/15	0	0	165,2 ms	2.165 ms	2/2	Funcional sob estresse
Recuperação	10/10	0	0	5,4 ms	6 ms	2/2	Recuperação total

A linha mais importante da tabela é a coluna **error** : ela é zero em todos os experimentos. Em nenhum cenário de falha o cliente recebeu erro. O sistema ou serviu normalmente, ou serviu mais devagar, ou degradou de forma explícita e controlada — que é exatamente a definição operacional de resiliência adotada neste trabalho.

4.2. Métricas de infraestrutura

Métrica	Baseline	NetworkChaos	PodChaos	StressChaos	Recuperação
CPU Gateway (HPA)	1m (1%)	2m (1%)	2m (1%)	2m (1%)	2m (1%)
CPU Serviço (HPA)	1m (0%)	1m (0%)	23m (15%)	27m (18%)	248m (165%)
Memória Gateway	54%	54%	55%	54%	54%
Memória Serviço	46%	46%	46%	46%	48%
Réplicas Gateway / Serviço	2 / 2	2 / 2	2 / 2	2 / 2	2 / 2

Os 248m (165%) na coluna "Recuperação" **não indicam carga durante a recuperação** — são o eco defasado do StressChaos atravessando o pipeline do metrics-server, conforme analisado em 3.3. É a evidência central da explicação sobre o HPA.

4.3. Eficácia por mecanismo

Mecanismo	Exercitado em	Evidência	Veredito
Timeout (2s)	NetworkChaos	Latência limitada; sem requisição presa nos 3000ms injetados	✓ Eficaz
Retry (3×, backoff 0,5s)	PodChaos	p95 de 508ms = assinatura do backoff; 15/15 ok	✓ Eficaz
Fallback (degraded , HTTP 200)	NetworkChaos	15 degradadas, 0 erros ao cliente	✓ Eficaz
Réplicas + readiness probe	PodChaos	Pod removido do balanceamento; ~1 requisição afetada de 15	✓ Eficaz
Limites de recurso (cgroup)	StressChaos	Estressor confinado; Gateway e monitoramento intactos	✓ Eficaz
Cache Redis	Todos	Baseline a 5,87ms com processamento de 100ms	✓ Eficaz (com ressalva da seção 5)
Circuit Breaker	NetworkChaos	Permaneceu CLOSED apesar de 15 falhas	✗ Não demonstrado — estado fragmentado entre 4 processos (ver 3.1)
HPA	StressChaos	Não escalou; carga só visível após o ataque	✗ Não demonstrado — janela menor que a defasagem das métricas (ver 3.3)

Seis dos oito mecanismos foram exercitados e comprovados sob falha real. Os dois que não foram falharam **por razões diagnosticadas e distintas**: o Circuit Breaker por um defeito de arquitetura da aplicação (estado local em ambiente multiprocesso), e o HPA por um defeito de desenho do experimento (duração insuficiente). Nenhum dos dois é um defeito do mecanismo em si.

5. Ameaças à Validade

Esta seção registra, com rigor, as limitações que restringem a generalização dos resultados.

1. **pod-failure não é pod-kill**. O manifesto usa `action: pod-failure`, que torna o pod indisponível substituindo seu container por um *pause*, sem destruí-lo. A queda abrupta de instância (`action: pod-kill`, que remove o pod e força reagendamento) **não foi coberta**. São modos de falha distintos: o `pod-kill` exercita adicionalmente o reagendamento e o tempo de startup, que este experimento não mediu. Observação: o `metadata.name` do recurso é `pod-kill-servico`, o que diverge da ação efetivamente configurada e deve ser renomeado.
2. **O baseline é um estado de cache quente, não de processamento**. Com `PROCESSING_DELAY_MS=100` e TTL de 30s, os 5,87ms do baseline medem essencialmente a latência de leitura do Redis. Comparações do tipo "410× mais lento" contrastam *cache hit* contra *cache miss sob ataque*, e não estado normal contra estado degradado. O cache foi limpo antes do NetworkChaos e do PodChaos, mas **não antes do StressChaos** — portanto parte dos 165ms

daquele experimento é atribuível a *cache miss* (que custa 100ms por si só), e não à contenção de CPU. A magnitude do efeito do estresse está, nessa medida, superestimada.

3. **O Serviço não consome CPU por construção.** Sendo o processamento um `time.sleep`, que libera o processador, o StressChaos tem pouco recurso a disputar. O experimento de saturação é, portanto, estruturalmente conservador: mede o efeito do estressor sobre um processo ocioso.
4. **Contadores do Prometheus inutilizáveis para análise quantitativa.** Conforme a seção 2.1, `gateway_retries_total`, `gateway_errors_total` e `gateway_forward_requests_total` oscilam entre workers e não podem sustentar afirmações numéricas. As análises deste relatório não dependem deles.
5. **A injeção do caos não é verificada pela automação.** Em `run-experiments.ps1`, as três chamadas `kubectl apply -f chaos/*.yaml 2>$null | Out-Null` descartam saída e código de erro. O script confirma que o `chaos-daemon` está em execução (e aborta caso não esteja), mas **não verifica se o objeto de caos foi criado nem se atingiu o estado `Injected`**. Nesta execução há evidência independente de que os três ataques ocorreram (latência de 2.408ms; CPU do Serviço em 79m/47m; elevação de CPU no PodChaos), mas a automação, como está, não distingue "o sistema resistiu ao ataque" de "o ataque nunca foi aplicado" — e essa distinção é a premissa de todo Chaos Engineering. **Correção necessária:** validar o exit code do `kubectl apply` e aguardar `status.conditions[type=AllInjected]=True` antes de gerar carga.
6. **Amostra de 15 requisições por experimento.** Adequada para caracterização qualitativa dos mecanismos; insuficiente para inferência estatística. p95 sobre 15 amostras é, na prática, o segundo maior valor observado.
7. **Ausência de grupo de controle.** Não foi executada uma variante *naive* (sem CB, retry, timeout e HPA) para isolar a contribuição de cada mecanismo. A aplicação **já possui a instrumentação necessária** para isso — o endpoint `/admin/toggle` permite desligar CB, retry e timeout em tempo de execução —, mas o experimento comparativo não foi conduzido.
8. **Ambiente local (Kind).** Cluster de nó único físico, sem latência de rede real entre nós, sem contenção de vizinhos e sem carga concorrente de produção.

6. Plano de Correções Priorizado

Consolidação das ações corretivas identificadas, ordenadas por impacto sobre a resiliência.

#	Correção	Origem	Camada
1	Mover o estado do Circuit Breaker para o Redis (ou reduzir para <code>--workers 1</code>)	Exp. 1	Código
2	Zerar <code>failure_count</code> em todo sucesso com estado CLOSED	Exp. 1	Código
3	Habilitar o modo multiprocess do <code>prometheus_client</code> (<code>PROMETHEUS_MULTIPROC_DIR</code>)	Metodologia	Código/Deploy
4	Validar a injeção do caos (exit code + <code>AllInjected</code>) antes de gerar carga	Metodologia	Automação
5	Elevar a duração do StressChaos para $\geq 180s$ e amostrar o HPA a cada 5s	Exp. 3	Experimento
6	Substituir <code>time.sleep</code> por trabalho de CPU real no Serviço	Exp. 3	Código
7	Adicionar experimento com <code>action: pod-kill</code> ; renomear o recurso atual	Exp. 2	Caos
8	Adicionar <code>PodDisruptionBudget</code> (<code>minAvailable: 1</code>)	Exp. 2	Infra
9	Executar a variante <i>naive</i> via <code>/admin/toggle</code> como grupo de controle	Validade	Experimento
10	Fazer o fallback reler o cache antes de retornar resposta estática	Arquitetura	Código

7. Conclusão

A aplicação foi submetida a três classes de falha — rede, instância e recurso — e **em nenhuma delas devolveu erro ao cliente**. Esse é o resultado central: 0 requisições com erro em 55 requisições enviadas ao longo de cinco fases experimentais.

Cada classe de falha foi absorvida por uma camada distinta de defesa, e a evidência permite atribuir o mérito com precisão em vez de creditá-lo genericamente ao conjunto:

- **Falha de rede** foi absorvida pelo **timeout somado ao fallback**: 15 requisições degradaram explicitamente com HTTP 200, nenhuma travou nos 3000ms injetados e nenhuma virou erro.
- **Falha de instância** foi absorvida pela tríade **réplicas + readiness probe + retry**: o pod defeituoso saiu do balanceamento em segundos e o retry recuperou a única requisição apanhada na janela de transição — cujo custo de 508ms é a assinatura mensurável do backoff de 500ms.
- **Falha de recurso** foi contida pelos **limites de cgroup**: o estressor ficou confinado aos pods do Serviço, e Gateway, Redis e a stack de monitoramento seguiram intactos.

Dois mecanismos não puderam ser demonstrados, e o diagnóstico de cada um é, em si, um resultado do trabalho. O **Circuit Breaker** está corretamente implementado, mas sua eficácia é anulada pela topologia de execução: com 2 réplicas $\times$ 2 workers gunicorn, existem quatro breakers independentes com contadores separados, e o limiar de 3 falhas torna-se até 12 no agregado. É um caso exemplar de um mecanismo de tolerância a falhas correto no papel e inerte na prática — precisamente o tipo de descoberta que só a injeção de falhas revela, já que nenhum teste unitário do breaker jamais falharia. O **HPA** não escalou porque a janela de ataque (45s) foi menor que a defasagem do pipeline de métricas ($\sim 15s$ de resolução + 30s de janela de estabilização); a prova está no registro de 248m (165% do alvo) na fase de recuperação — a carga existiu, mas só se tornou visível depois que o estressor já havia sido removido.

O aprendizado metodológico mais duradouro, porém, veio da própria instrumentação. A descoberta de que os contadores do Prometheus **decresem** ao longo da execução (67 → 63 → 83) — impossível para um contador monotônico — expôs que as métricas da aplicação eram lidas de workers aleatórios e não descreviam o sistema, mas um quarto dele. Isso obrigou a reconstruir a análise sobre a medição feita no cliente. A lição é que **observabilidade não é a presença de um dashboard: é a garantia de que o número exibido corresponde ao sistema observado**. Um experimento de caos apoiado em métricas silenciosamente erradas produz conclusões confiantes e falsas — e vale notar que o mesmo padrão de cegueira aparece na automação, que declarava "Todos experimentos aplicados!" mesmo quando todos os `kubect1 apply` haviam falhado.

Em síntese: a defesa em profundidade funcionou, cada camada cobriu a falha para a qual foi projetada, e as duas lacunas encontradas são acionáveis e estão priorizadas na seção 6.

8. Referências

1. ROSENTHAL, C.; JONES, N. *Chaos Engineering: System Resiliency in Practice*. O'Reilly Media, 2020.
 2. BASIRI, A. et al. Chaos Engineering. *IEEE Software*, v. 33, n. 3, p. 35–41, 2016.
 3. NYGARD, M. *Release It!: Design and Deploy Production-Ready Software*. 2. ed. Pragmatic Bookshelf, 2018. (origem do padrão Circuit Breaker)
 4. BEYER, B. et al. *Site Reliability Engineering: How Google Runs Production Systems*. O'Reilly Media, 2016. Disponível em: <https://sre.google/sre-book/table-of-contents/>
 5. NETFLIX. *Chaos Monkey*. Disponível em: <https://netflix.github.io/chaosmonkey/>
 6. CHAOS MESH. *Documentation*. Disponível em: <https://chaos-mesh.org/docs/>
 7. KUBERNETES. *Horizontal Pod Autoscaling*. Disponível em: <https://kubernetes.io/docs/tasks/run-application/horizontal-pod-autoscaler/>
 8. PROMETHEUS. *Python client — multiprocess mode*. Disponível em: https://github.com/prometheus/client_python
-

9. Reprodução

```
# 1. Cluster Kind (3 nós)
kind create cluster --config kind-config.yaml

# 2. Build e carga das imagens
docker build -t gateway:latest -f docker/gateway.Dockerfile .
docker build -t servico:latest -f docker/servico.Dockerfile .
kind load docker-image gateway:latest
kind load docker-image servico:latest

# 3. Aplicação
kubectl apply -f k8s/namespace.yaml
kubectl apply -f k8s/

# 4. Chaos Mesh (via Helm – necessário para o containerd do Kind)
helm repo add chaos-mesh https://charts.chaos-mesh.org
helm upgrade --install chaos-mesh chaos-mesh/chaos-mesh \
  -n chaos-mesh --create-namespace \
  --set chaosDaemon.runtime=containerd \
  --set chaosDaemon.socketPath=/run/containerd/containerd.sock

# 5. metrics-server (necessário para o HPA)
kubectl apply -f https://github.com/kubernetes-sigs/metrics-server/releases/latest/download/component
kubectl patch deployment metrics-server -n kube-system --type='json' \
  -p='[{"op":"replace","path":"/spec/template/spec/containers/0/args","value":["--cert-dir=/tmp","--s

# 6. HPA e monitoramento
kubectl apply -f k8s/hpa.yaml
kubectl apply -f k8s/grafana-secret.yaml
kubectl apply -f k8s/monitoring/

# 7. Verificação
curl http://localhost:30000/api/data           # aplicação
kubectl get hpa -n trabalho-sis-dis           # HPA
kubectl top nodes                             # metrics-server
kubectl get pods -n chaos-mesh -l app.kubernetes.io/component=chaos-daemon

# 8. Bateria completa de experimentos
.\run-experiments.ps1                         # gera monitoring-report.json
```

Acessos: Aplicação <http://localhost:30000> · Prometheus <http://localhost:30090> · Grafana <http://localhost:30030>

Apêndice A — Dados Brutos Coletados

Transcrição integral das medições da execução de **2026-07-15 13:30:32**, geradas por `run-experiments.ps1` e preservadas em `monitoring-report.json` no repositório. Os valores citados ao longo das seções 3 e 4 derivam exclusivamente destes registros.

A.1. Baseline — estado estável, sem ataque

```
request_results: { ok: 15, degraded: 0, error: 0, total: 15, avg_ms: 5.87, p95_ms: 6 }
circuit_breaker: CLOSED, failures = 0
gateway_errors:   {}                                (nenhum erro registrado)
gateway_retries:  {}                                (nenhuma tentativa)
gateway_forwards: { success: 67 }
HPA gateway-hpa:  current=2 desired=2 | cpu=1m (1%)  memory=71892Ki (54%)
HPA servico-hpa:  current=2 desired=2 | cpu=1m (0%)  memory=61308Ki (46%)
pods: gateway=2, servico=2, total=8
```

A.2. NetworkChaos — latência 3000ms + perda 30%

```
request_results: { ok: 0, degraded: 15, error: 0, total: 15, avg_ms: 2408.13, p95_ms: 7514 }
circuit_breaker: CLOSED, failures = 2                                (worker amostrado - ver secao 2.1)
gateway_errors:   { request_error: 21, fallback: 10, timeout: 6 }
gateway_retries:  { error: 21, timeout: 6 }
gateway_forwards: { success: 63, error: 21, timeout: 6 }
HPA gateway-hpa:  current=2 desired=2 | cpu=2m (1%)  memory=71948Ki (54%)
HPA servico-hpa:  current=2 desired=2 | cpu=1m (0%)  memory=61340Ki (46%)
```

A.3. PodChaos — pod-failure, mode:one, 30s

```
request_results: { ok: 15, degraded: 0, error: 0, total: 15, avg_ms: 46, p95_ms: 508 }
circuit_breaker: CLOSED, failures = 2                                (worker amostrado)
gateway_errors:   { request_error: 20, fallback: 9, timeout: 3 }
gateway_retries:  { error: 20, timeout: 3 }
gateway_forwards: { success: 83, error: 20, timeout: 3 }
HPA gateway-hpa:  current=2 desired=2 | cpu=2m (1%)  memory=72150Ki (55%)
HPA servico-hpa:  current=2 desired=2 | cpu=23m (15%) memory=61032Ki (46%)
pods antes: gateway=2, servico=2, total=8
pods depois: gateway=2, servico=2, total=8
```

A.4. StressChaos — CPU 2 workers @ 100%, mode:all, 45s

```
request_results: { ok: 15, degraded: 0, error: 0, total: 15, avg_ms: 165.2, p95_ms: 2165 }
circuit_breaker: CLOSED, failures = 0
gateway_errors:   { request_error: 19, fallback: 10, timeout: 6 }
gateway_retries:  { error: 19, timeout: 6 }
gateway_forwards: { success: 87, error: 19, timeout: 6 }
HPA gateway-hpa:  current=2 desired=2 | cpu=2m (1%)  memory=72022Ki (54%)
HPA servico-hpa:  current=2 desired=2 | cpu=27m (18%) memory=61322Ki (46%)
pods antes: gateway=2, servico=2 | pods depois: gateway=2, servico=2 (sem escalonamento)

CPU por pod - rate(container_cpu_usage_seconds_total[1m]), em cores:
servico-c684ffb84-dcp7p ..... 0.0789 (79m) <-- alvo do ataque
servico-c684ffb84-9pqcc ..... 0.0465 (47m) <-- alvo do ataque
gateway-7df6dd8599-h8htk ..... 0.0014 (1.4m) <-- nao afetado
gateway-7df6dd8599-cp47z ..... 0.0013 (1.3m) <-- nao afetado
prometheus-6c8cd969dc-w2c6t ..... 0.0084 (8.4m)
grafana-7ff7d48c99-wrftk ..... 0.0080 (8.0m)
redis-55f8784fcf-ntcjb ..... 0.0032 (3.2m)
redis-exporter-664cd4dcc-wf2zg ... 0.0021 (2.1m)
```


Este bloco é a evidência de duas afirmações da seção 3.3: o ataque **foi injetado** (pods do Serviço a 79m e 47m contra ~1m no baseline) e ficou **confinado ao cgroup dos pods-alvo** (Gateway, Redis, Prometheus e Grafana inalterados).

A.5. Recuperação — pós-experimentos, sem ataque ativo

```
request_results: { ok: 10, degraded: 0, error: 0, total: 10, avg_ms: 5.4, p95_ms: 6 }
circuit_breaker: CLOSED, failures = 0
gateway_errors: { request_error: 19, fallback: 12, timeout: 6 }
gateway_retries: { error: 19, timeout: 6 }
gateway_forwards: { success: 96, error: 19, timeout: 6 }
HPA gateway-hpa: current=2 desired=2 | cpu=2m (1%)      memory=72028Ki (54%)
HPA servico-hpa: current=2 desired=2 | cpu=248m (165%)  memory=63574Ki (48%)
```

Os **248m (165%)** registrados aqui, com o StressChaos já removido e o sistema em repouso (10/10 ok a 5,4ms), são o eco defasado do ataque atravessando o pipeline do metrics-server. É a evidência central da análise do HPA na seção 3.3.

A.6. Demonstração da não-monotonicidade dos contadores

Evolução da série `gateway_forward_requests_total{status="success"}` ao longo das cinco fases, extraída de A.1–A.5:

Fase	Baseline	NetworkChaos	PodChaos	StressChaos	Recuperação
<code>forwards{success}</code>	67	63	83	87	96
<code>errors{request_error}</code>	0	21	20	19	19
<code>errors{fallback}</code>	0	10	9	10	12

As células em negrito são decréscimos. Um contador Prometheus é monotonicamente crescente por definição — um contador real **não pode** cair de 67 para 63. Estes decréscimos são a prova empírica, colhida no próprio experimento, de que scrapes sucessivos leram workers gunicorn distintos, cada um com seu registro de métricas em memória. É o fundamento da ressalva metodológica da seção 2.1 e da correção nº 3 da seção 6.